



Program: BS Computer Science

Semester: Fall 2025

Credit Hour(s): 1

Pre-requisite(s): EE1005 Digital Logic Design

Instructor: Engr. Muhammad Hassaan Bin Shoukat – hassaan.shoukat@nu.edu.pk

Lab # 2

Intel iAPX 88 Architecture & Assembly Fundamentals

1. Objectives

By the end of this lab, students will be able to:

- Understand the **Intel iAPX 88 architecture**.
- Identify and explain the **registers** and their roles.
- Explore different **instruction groups** of 8086 assembly language.
- Understand the **segmented memory model**.
- Write, execute and explain **first assembly program**.

2. Intel iAPX 88: History

- Intel introduced the **8086 microprocessor** in **1978** as a 16-bit processor.
- The **Intel 8088 (iAPX 88)** followed in **1979**, designed for IBM PCs and became the CPU for the first IBM PC.
- Difference:
 - 8086 → **16-bit external data bus**.
 - 8088 → **8-bit external data bus**, making it cheaper and more compatible with existing 8-bit hardware.

3. Register Architecture

Registers are *small, fast storage units* inside the CPU used for processing data and addresses.

8086/8088 has **14 registers** of **16-bit** wide, grouped as:

A. **General Purpose Registers (GPRs)**

- **AX** (Accumulator – AH/AL) – arithmetic, logic, I/O operations
- **BX** (Base – BH/BL) – used as a pointer to memory locations, frequently works with *DS*
- **CX** (Counter – CH/CL) – loops, string and shifts. CL is often used in shift/rotate instructions
- **DX** (Data – DH/DL) – multiplication/division, I/O. Works with AX to hold large results

Each GPR can be accessed as high/low 8-bit parts (e.g., AH/AL).



B. Segment Registers:

The iAPX88 uses a segmented memory model to address up to 1 MB of physical memory using 16-bit registers. Each segment register holds the starting address of a 64 KB memory segment.

- **CS** (Code Segment): Points to the beginning of the memory segment containing the *program's instructions*.
- **DS** (Data Segment): Points to the beginning of the memory segment containing the *program's data*.
- **SS** (Stack Segment): Points to the beginning of the memory segment used for the *stack operations* (function calls, local variables).
- **ES** (Extra Segment): An additional data segment register, often used for *string operations*.

C. Pointer and Index Registers

These registers are used to hold offset addresses for memory access.

- **SP** (Stack Pointer): It *always* works with SS (Stack Segment). Points to the **top of the stack**.
- **BP** (Base Pointer): It is used to access parameters and local variables passed to a subroutine from the stack. Works with SS.
- **SI** (Source Index): It is used as a pointer to source data in string operations. Works with DS.
- **DI** (Destination Index): It is used as a pointer to destination data in string operations.

D. Instruction Pointer

- Holds the **address/offset of the next instruction** to be executed within the current code segment.
- Always works with **CS** (Code Segment).
- Together **CS:IP** form the complete address of the next instruction.

E. **Flags Register:** A 16-bit register where each bit (flag) indicates the status of the CPU after an instruction is executed.

Important Status Flags

- **CF** (Carry Flag) → set if arithmetic carry/borrow occurs.
- **ZF** (Zero Flag) → set if result is zero.
- **SF** (Sign Flag) → set if result is negative.
- **OF** (Overflow Flag) → set if signed overflow occurs.
- **PF** (Parity Flag) → set if result has even number of 1's.
- **AF** (Auxiliary Carry) → set for BCD operations.

Control Flags

- **IF** (Interrupt Enable Flag) → enables/disables interrupts.
- **DF** (Direction Flag) → controls string operation direction (SI/DI).
- **TF** (Trap Flag) → single-step debugging.



4. Instruction Groups

Instructions are classified as:

- **Data Transfer Instructions** – MOV, PUSH, POP
- **Arithmetic Instructions** – ADD, SUB, MUL, DIV, INC, DEC
- **Logical Instructions** – AND, OR, XOR, NOT
- **Bit-Manipulation Instructions** – SHL, SHR
- **Control Transfer Instructions** – JMP, CALL, RET, LOOP
- **Processor Control Instructions** – HLT, WAIT, NOP

5. Segmented Memory Model

- The Intel 8086/8088 can address up to **1 MB of physical memory**.
- However, its registers are only **16 bits** wide, so a single register can address only **64 KB** (2^{16} bytes).
- To overcome this limitation, Intel introduced the **segmented memory model**.
- In this model, a **physical address** is formed by combining:
 - A **segment address** (from CS, DS, SS, ES)
 - An **offset address** (from IP, BX, SI, DI, etc.)
 - $Physical\ Address = (Segment\ Register \times 16) + Offset\ Register$
- The result we got from the above formula will be a 20-bit physical address.

6. Assembly Language Fundamentals

Basic elements of assembly programs:

- **Mnemonics** → MOV, ADD, JMP (human-readable form of instructions).
- **Instructions** → These are the commands that the CPU can execute. They often consist of an opcode (operation code) and one or more operands.
 - **Opcode**: Specifies the operation to be performed (e.g., MOV for move, ADD for add).
 - **Operands**: The data or memory addresses the instruction will operate on.
- **Directives** → Commands for the assembler, not the CPU. They tell the assembler how to process the program (e.g., ORG, DB, DW).
- **Comments** → Used for writing about the code. It begins with (;).
- **Labels** → Symbolic names used to represent a memory address. It is used as jump destinations.



7. Lab Tasks

1. Trace the execution of the given assembly code and record the values of the **AX** and **BX** registers after each instruction that modifies them. Use a table to present your findings.

Instruction	Value of AX	Value of BX	Value of CX
mov ax, 5			
mov bx, 10			
add ax, bx			
mov cx, 15			
add cx, bx			

2. Write an assembly program that adds three numbers, 7, 10 and 20, and stores the result in **CX** register.
3. Write an assembly code that perform the following, and store in **DX** register:
 $70 + 41 + 85$
4. Write an assembly language program to perform following: $100 - 8 - 22$
5. Write the code for adding two numbers, then subtract it with the third one and store the result in the fourth register.